

순환 신경망(RNN), 장단기 메모리(LSTM)

Word2Vec

- 2013 구글에서 공개한 임베딩 모델 → 단어 간 유사성 측정 위해 **분포 가설(distributional hypothesis)** 기반 개발
- **분포 가설 : 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다.**
 - 단어 간의 동시 발생(co-occurrence) 확률 분포를 이용해 단어 간 유사성을 측정
 - (ex) '내일 자동차를 타고 부산에 간다', '내일 비행기를 타고 부산에 간다'라는 두 문장에서 '자동차'와 '비행기'는 주변에 분포한 단어들이 동일하거나 유사하므로 비슷한 의미를 가질 것이라 예상
- 이런 가정을 통해 단어의 **분산 표현(Distributed Representation)**을 학습할 수 있음
- **분산 표현 : 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것** → 분포 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남
 - = 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 됨
 - (ex) '자동차'와 '비행기'라는 단어는 벡터 공간에서 서로 가까운 위치에 표현됨
- 이러한 방법으로 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계 극복
- 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고, 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보를 효과적으로 표현
- 이러한 분산 표현 방식은 다양한 자연어 처리 작업에서 높은 성능을 보여주면, downstream 작업에서 더 뛰어난 성능

단어 벡터화

1. 희소 표현(Sparse representation)

- 원-핫 인코딩, TF-IDF

표 6.3 단어의 희소 표현

소	0	1	0	0	0
일고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- 대부분의 벡터 요소가 0으로 표현되는 희소 표현 방법
- 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비
- 단어간의 유사성을 반영하지 못하고, 벡터 간 유사성을 계산하는 데도 많은 비용 발생

2. 밀집 표현(dense representation)

- Word2Vec

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
일고	-0.2893	0.3814	...	-0.1492	-0.2814
외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- 고정된 크기의 실수 벡터로 표현 → 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지 않음
- 벡터 공간상에서 단어 간의 거리를 효과적으로 계산 가능
- 벡터의 대부분이 0이 아닌 실수로 이루어져 있어 효율적으로 공간 활용 가능

- 학습을 통해 벡터화하기 때문에 단어의 의미 비교 가능
- 밀집 표현된 벡터를 단어 임베딩 벡터(Word Embedding Vector)라고 하며, Word2Vec은 대표적인 단어 임베딩 기법
 - Word2Vec은 밀집 표현을 위해 CBoW & Skip-gram이라는 2가지 방법 사용

CBoW(Continuous Bag of Words)

- 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법
 - 중심단어(Center Word): 예측해야할 단어
 - 주변단어(Context Word) : 예측에 사용되는 단어
- 윈도우(Window) : 중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지를 정해야하는 데 이 범위를 의미
 - 윈도우를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여 마지막 단어까지 학습
- 슬라이딩 윈도우(Sliding Window) : 학습을 위해 윈도우를 이동해 가며 학습
- CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어 학습

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

- 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터의 구성
- 학습 데이터 (주변 단어 | 중심 단어)
- CBoW의 모델 구조 : 얻어진 학습 데이터는 다음 과 같은 인공 신경망을 학습하는 데 사용됨

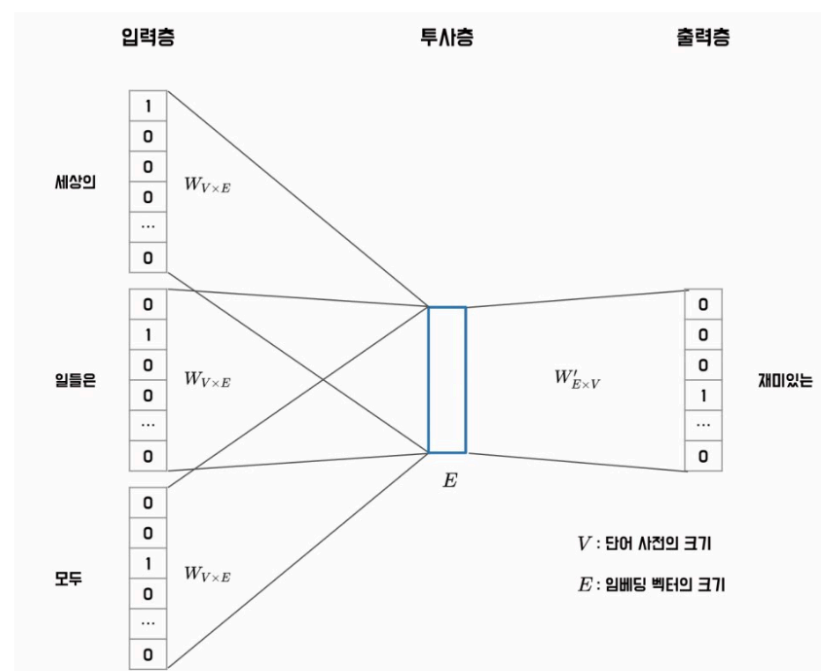


그림 6.5 CBoW 모델 구조

- 각 입력 단어의 원-핫 벡터를 입력값으로 받음
- 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측
- 입력 단어는 원-핫 벡터로 표현돼 Projection Layer에 입력됨
 - Projection Layer : 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표(Lookup table, LUT) 구조
 - Projection Layer를 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환됨
 - 입력된 세 단어의 임베딩 벡터를 V(세상의), V(일들은), V(모두)라고 가정하는 이 벡터의 평균값을 계산
- 계산된 평균 벡터를 가중치 행렬 $W'(E \times V)$ 와 곱하면 V크기의 벡터를 얻음

- 이 벡터에 Softmax 함수를 이용해 중심 단어 예측

Skip-gram

- 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델
- 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만들
 - 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습시킴

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

- **Skip-gram과 CBoW는 학습 데이터의 구성 방식에 차이가 있음**
 - CBoW : 하나의 윈도우에서 하나의 학습 데이터가 만들어짐
 - Skip-gram : 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어짐
 - 하나의 중심 단어를 통해 여러 개의 주변 단어를 예측하므로 → 더 많은 학습 데이터 세트 추출 가능 → CBoW 보다 더 뛰어난 성능을 보임
 - 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 되고 단어 벡터 공간에서 더 유의미한 거리 관계를 형성할 수 있음
- **Skip-gram 모델 구조**

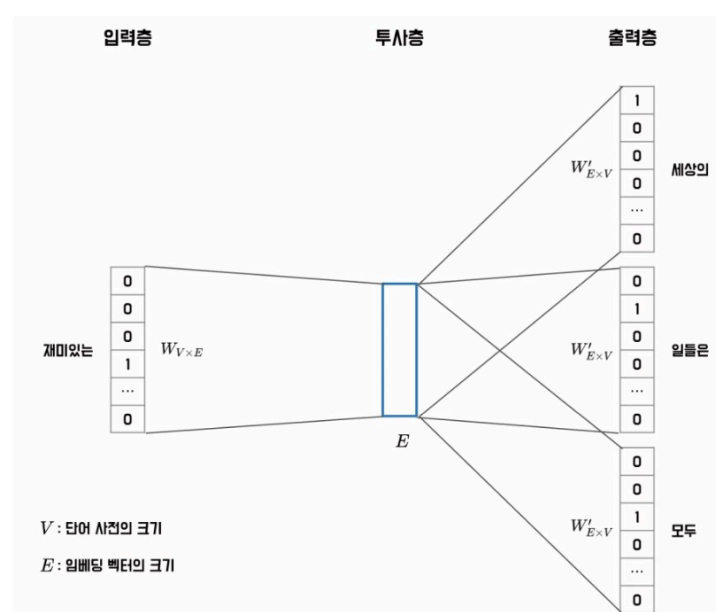


그림 6.7 Skip-gram 모델 구조

- 입력 단어의 원-핫 벡터를 Projection Layer에 입력하여 해당 단어의 임베딩 벡터를 가져옴
- 입력 단어의 임베딩과 $W'(E \times V)$ 가중치와의 곱셈을 통해 V 크기의 벡터를 얻음
- 이 벡터에 Softmax 연산을 취해 주변 단어를 예측
 - Softmax 연산은 모든 단어를 대상으로 내적 연산을 수행
 - [단점] 말뭉치 크기가 커지면 필연적으로 단어 사전의 크기도 커지므로 대량의 말뭉치를 통해 Word2Vec 모델을 학습할 때 학습 속도가 느려짐
 - [해결방법] 계층적 소프트맥스와 네거티브 샘플링 기법

계층적 소프트맥스(Hierarchical Softmax)

- 출력층을 Binary tree 구조로 표현해 연산을 수행
 - 자주 등장하는 단어일수록 트리의 상위 노드에 위치

- 드물게 등장하는 단어일수록 하위 노드에 배치
- 일반적인 Softmax 연산에 비해 빠른 속도와 효율성

• 계층적 소프트맥스 구조

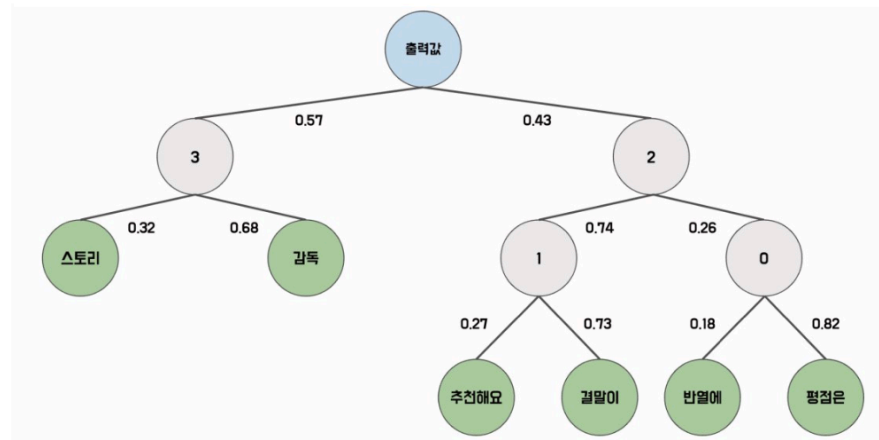


그림 6.8 계층적 소프트맥스 구조

- 각 노드는 학습이 가능한 벡터를 가짐
- 입력값은 해당 노드의 벡터와 내적값을 계산 → 시그모이드 함수를 통해 확률 계산
- Leaf Node : 각 단어를 의미
- 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
- 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있음
 - (ex) '추천해요' 단어는 $0.43 \times 0.74 \times 0.27 = 0.085914$ 의 확률 → 학습 시 1, 2번 노드의 벡터만 최적화하면 됨
- 단어 사전의 크기를 V라고 했을 때
 - 일반적인 소프트맥스 연산: $O(V)$ 의 시간 복잡도
 - 계층적 소프트맥스의 시간 복잡도: $O(\log_2 V)$ 의 시간 복잡도

네거티브 샘플링

- Word2Vec 모델에서 사용되는 확률적인 샘플링 기법 → 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용
- 학습 원도 내에 등장하지 않는 단어를 n개 추출하여 정답 단어와 함께 소프트맥스 연산 수행 → 전체 단어의 확률을 계산할 필요없이 모델을 효율적으로 학습
 - 추출할 단어 n : 일반적으로 5~20개 사용
 - 각 단어가 추출될 확률

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- 먼저 각 단어 w_i 의 출현 빈도수를 $f(w_i)$ 로 나타냄
 - (ex) 말뭉치 내에 단어 '추천해요'가 100번 등장 → 전체 단어의 빈도가 2,000이라면 → $f(\text{추천해요}) = 100/2000 = 0.05$
- $P(w_i)$: 단어 w_i 가 네거티브 샘플로 추출될 확률
 - 이때 출현 빈도수에 0.75 제곱한 값을 정규화 상수로 사용 → 실험을 통해 얻어진 최적값
- 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 아니면 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류
 - 이를 위해 로지스틱 회귀 모델을 사용 → 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습

• 네거티브 샘플링 모델의 훈련 데이터

입력 데이터	출력 데이터
재미있는, 세상의	1
재미있는, 일들은	1
재미있는, 모두	1
재미있는, 걸어가다	0
재미있는, 주십시오	0
재미있는, 미래	0
재미있는, 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

입력 데이터	출력 데이터	라벨
재미있는, 세상의	정답	1
재미있는, 일들은	정답	1
재미있는, 모두	정답	1
재미있는, 걸어가다	무관	0
재미있는, 주십시오	무관	0
재미있는, 미래	무관	0
재미있는, 안녕하세요	무관	0

- 앞의 3개는 실제 문맥에서 함께 등장하는 단어 쌍 (양성 샘플, label = 1)
- 뒤의 4개는 문맥상 관련 없는 단어 쌍 (음성 샘플, label = 0)
- 이런 식으로 양성(1)과 음성(0) 샘플을 만들어 이진 분류로 학습

• 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

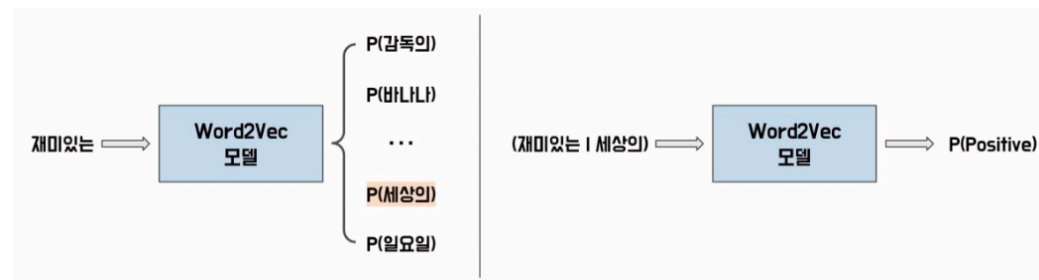


그림 6.10 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec

◦ 소프트맥스 기반 Word2Vec

- 입력 단어: 재미있는
- 출력 단어 전체에 대해 확률을 계산: P(감독의), P(바나나), ..., P(세상의), P(일요일)
- 이 중 "정답 단어(예: 세상의)"의 확률이 높아지도록 학습

⇒ 전체 단어에 대해 확률 분포를 계산해야 하므로 연산량이 많음 (특히 단어 수가 수만 개일 때)

◦ 네거티브 샘플링 기반 Word2Vec

- 입력 단어: 재미있는
- 정답 단어: 세상의 → label 1
- 샘플로 뽑은 무관 단어들: 바나나, 감독의, 일요일 → label 0
- 모델은 재미있는, 세상의 쌍이 positive인지(1) 판단하는 이진 분류를 학습

⇒ 연산량이 적고, 성능도 유사하게 잘 나와서 실제로 훨씬 자주 사용됨

fastText

- 2015년 메타의 FAIR 연구소에서 개발한 오픈소프 임베딩 모델 → 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘
- 단어와 문장을 벡터로 변환하는 기술을 기반으로 → 이리 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용됨
- 단어의 하위 단어를 고려하므로 Word2Vec 보다 더 높은 정확도와 성능 제공 → 하위 단어 고려위해 N-gram을 사용해 단어를 분해하고 벡터화하는 방법으로 동작

- fastText에서는 단어의 벡터화를 위해 <, > 와 같은 특수 기호를 사용 → 단어의 시작과 끝 표현 → 단어의 하위 문자열을 고려하는 데 중요한 역할
- 기호가 추가된 단어는 N-gram을 사용하여 하위 단어 집합(Subword set)로 분해됨
 - '서울특별시'를 바이그램으로 분해 → '서울', '울특', '특별', '별시'가 됨
 - 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함
 - 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 갖게 됨
 - 이러한 하위 단어 벡터들은 단어의 벡터 표현을 구성 → 이를 사용하여 자연어 처리 작업을 수행

• fastText의 하위 단어 집합

입력 단어	⟨ ⟩ 더하기	N-gram 분해	전체 단어 추가
서울특별시	⟨서울특별시⟩	⟨서울 서울특 울특별 특별시 별시⟩	⟨서울 서울특 울특별 특별시 별시⟩ ⟨서울특별시⟩

그림 6.11 fastText의 하위 단어 집합

• '서울특별시'라는 토큰에 대한 처리과정

1. 토큰 양 끝에 특수 기호 <, > 를 붙여 시작과 끝을 인식 가능하게 함 → '<서울특별시>'
 2. N-gram 분해
 - 예: 3-gram(=trigram)을 사용하면
 - ['<서울', '서울특', '울특별', '특별시', '별시>'] 등으로 하위 단어(subword) 분해됨
 3. 토큰 자체도 포함
 - '서울특별시' 라는 전체 단어도 함께 포함되어 하위 단어 집합을 구성
- 하위 단어 집합과 임베딩 계산
 - 위와 같은 방식으로 하위 단어 집합이 만들어짐
 - 각 하위 단어는 고유한 임베딩 벡터를 가짐
 - 전체 단어의 벡터는 하위 단어들의 임베딩 벡터를 합산하여 평균을 내어 계산됨 → 즉, 단어의 최종 임베딩 = 하위 단어 임베딩의 평균
 - fastText의 일반적 특징
 - 다양한 N-gram을 적용하여 하위 단어 벡터를 구성
 - 단어의 부분 문자열(substring)도 고려하므로 유연하고 정교한 임베딩 학습 가능
 - 단어끼리 정보 공유
 - 같은 하위 단어를 공유하는 단어들끼리는 임베딩 정보를 공유 → 예: '정보처리', '정보보호' 등
 - 이를 통해 유사한 단어끼리 비슷한 임베딩을 갖게 되어 단어 간 의미 유사도 향상
 - OOV(Out-of-Vocabulary) 단어 처리 가능
 - 사전에 없던 단어도 하위 단어 단위로 나눠 임베딩 계산 가능
 - 예: '개인택시', '정보처리기사', '임대차보호법' → '개인정보보호법' 과 연관된 하위 단어를 포함
 - → 이 단어들의 하위 단어 임베딩을 통해 '개인정보보호법' 과 의미적으로 연관된 임베딩 계산 가능

순환 신경망 (Recurrent Neural Network, RNN)

- RNN은 순서가 있는 연속형 데이터(sequence data)를 처리하는 데 적합한 구조
- 각 시점(time step)의 데이터가 이전 데이터와 독립적이지 않기 때문에 RNN이 효과적임
- 예시
 - 3월 7일 주가는 3월 6일 주가에 영향을 받을 가능성이 높음 → 연속형 데이터
 - 자연어 데이터도 연속형 데이터로, 단어 간 문맥 관계를 통해 다음 단어를 예측할 수 있음

금요일이 지나면 _
수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면 ...

- 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추
 - 첫번째 문장 : ‘금요일이 지나면’이라는 문장을 보고 ‘주말’ 또는 ‘토요일’ 등의 단어가 등장할 것이라고 예측
 - 두번째 문장 : 앞선 단어 패턴을 고려했을 때 ‘주말’ 보다는 ‘토요일’이 더 자연스러운 예측
- 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미를 형성
 - 자연어는 연속형 데이터의 특성 → t번째 단어는 t-1번째 가지의 단어에 영향을 받아 결정
- 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계 (Correlation)존재
 - ‘금요일이 지나면’이라는 문장에서 ‘지나면’이라는 단어는 ‘금요일’이라는 앞선 문맥과 상호작용해 전체적인 의미를 형성
 - 자연어 처리에서는 이러한 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요

RNN의 작동 방식

- 이전 은닉 상태와 현재 입력을 같이 활용해 현재 은닉 상태 계산
- 시계열 예측, 자연어 처리(NLP), 음성 인식 등에서 주로 사용됨
- 현재 시점 출력은 이전 정보에 기반
- 순환 신경망은 연속형 데이터를 순서대로 입력받아 처리
 - 각 시점마다 은닉 상태(Hidden state)의 형태로 저장
 - 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드 : 순환 신경망의 셀(Cel)

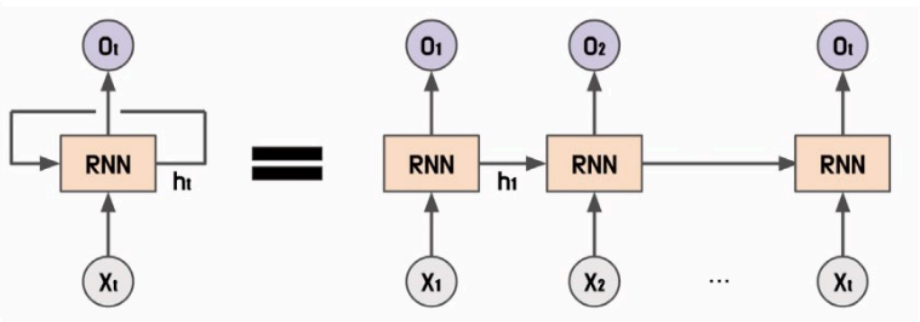


그림 6.12 순환 신경망의 셀

아주 중요한 특징 → 모든 시간 단계에서 동일한 가중치 사용 (reused at every time step)
RNN의 큰 장점 중 하나는 → 모든 시간 단계에서 같은 가중치와 같은 계산 과정을 사용한다는 것
Parameter sharing across time → 시간이 흘러도 동일한 파라미터로 처리함으로써 학습 효율이 좋고, 데이터가 많아도 파라미터 수가 폭발적으로 늘어나지 않음

일반적인 Fully Connected 신경망은?
만약 일반적인 신경망(FC)을 사용해서 문장을 처리한다고 가정하면 단어가 하나 늘어날 때마다 그 단어를 위한 새로운 가중치(weight)가 필요
즉, 입력의 길이가 늘어나면 가중치도 계속 늘어남 → 데이터가 많아질수록 모델 크기 커짐 → 비효율적

RNN은 다르다 → Parameter Sharing (가중치 공유)
RNN에서는 시간에 따라 입력되는 데이터가 다르더라도 → 계산 방식과 사용하는 가중치(W_{hx} , W_{hh} , W_{hy})는 항상 동일
예를 들어
시간 t=1 → W_{hx} , W_{hh} , W_{hy} 사용
시간 t=2 → 똑같이 W_{hx} , W_{hh} , W_{hy} 사용
시간 t=100 → 여전히 W_{hx} , W_{hh} , W_{hy} 사용
이렇게 하면 → 입력 시퀀스가 아무리 길어져도 필요한 가중치의 수는 늘어나지 않음

즉, 모델 파라미터 수가 폭발적으로 늘어나지 않음 → 학습과 저장이 훨씬 효율적임.
모델이 훨씬 간결해짐 → 긴 데이터도 효율적으로 처리 가능
학습 데이터가 적어도 괜찮음 → 매번 새로운 파라미터를 학습할 필요가 없어서 더 빠르게 학습 가능
일반화(generalization)도 잘 됨 → 똑같은 가중치를 다양한 시간에서 사용하기 때문에 일관성 있게 패턴을 배움

- 주요 수식

- 은닉 상태 계산: $h_t = \varphi(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

- 출력 계산: $y_t = \varphi(W_{hy} h_t + b_y)$

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy} h_t + b_y)$$

- φ 는 비선형 활성화 함수 (tanh, ReLU 등)

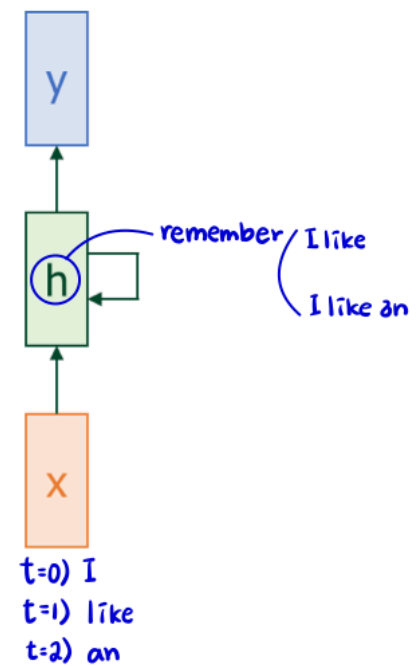
■ Basic RNN Architecture

RNN은 입력을 시간 순서대로 하나씩 처리하면서 hidden state(숨겨진 상태)를 유지
여기서 hidden state란 바로 → 과거의 정보를 담아두는 메모리(memory) 역할
즉, 과거에 어떤 입력이 들어왔는지 기억하는 장치
RNN은 매 시점(time step)마다 새로운 입력을 받고
→ 이전까지의 hidden state와 새 입력을 합쳐서
→ 새로운 hidden state와 출력(output)을 만들어 냄

RNNs process input one time step at a time, maintaining a hidden state

- A hidden state acts as **memory**
- **At each time step t :**
 - Given:
 - x_t : input at time t 현재 시간 t에서의 입력 데이터
 - h_{t-1} : hidden state from previous time step
 - Compute: 이전 시간의 hidden state(기억하고 있던 정보)
 새로운 hidden state 계산 $h_t = \text{tanh}(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$
비선형성 현재입력 이전 은닉상태
 출력 계산 $y_t = W_{hy}h_t + b_y$
 - Where:
 - W_{hx}, W_{hh}, W_{hy} : weight matrices 학습되는 가중치 행렬
 - b_h, b_y : biases 학습되는 편향
- The same weights are **reused at every time step**
 - Parameter sharing across time

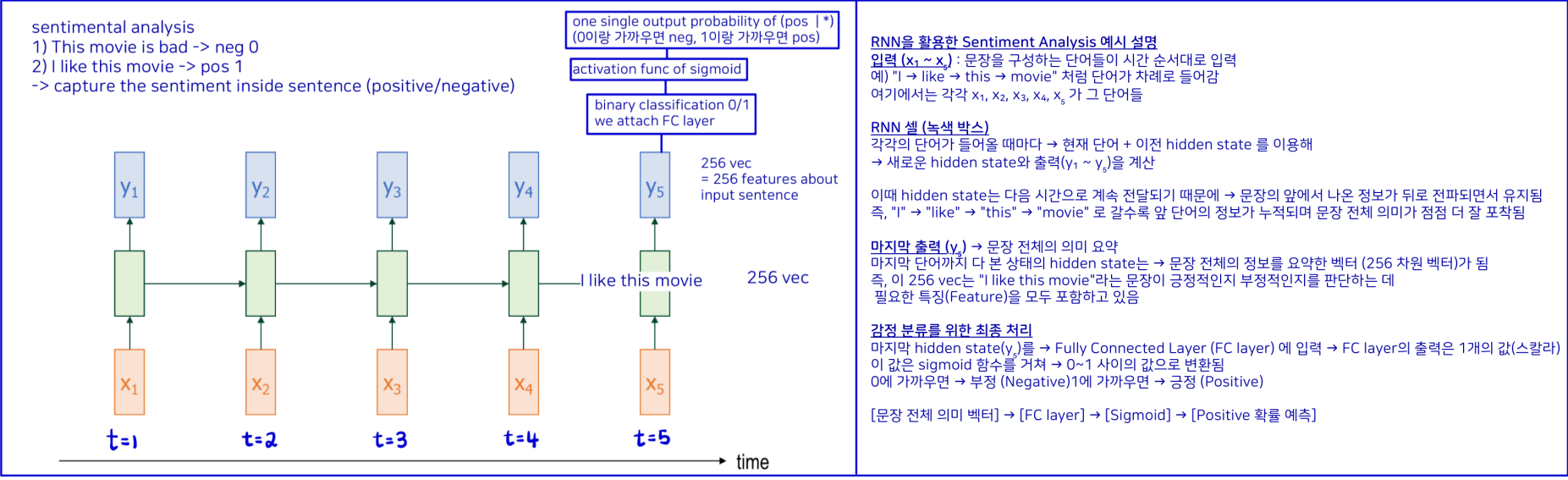
learnable parameter
학습을 통해서 최적화되며,
RNN 전체에서 공유됨



RNN의 timestamp는 각 timestamp마다 하나의 vector를 입력 받음 우리는 여러 개의 타임스텝을 가지고 있고, 각 타임스텝마다 하나의 벡터를 입력으로 받음

이 입력은 가중치 행렬(weight matrix)을 사용해 처리되며, 처리(processing)된다는 것은 곱셈(multiple)을 의미
즉, 벡터(vector)와 행렬(matrix)의 곱셈이 일어나며, 은닉 상태(hidden state)를 생성

이 hiddenstate(y_t)도 벡터이며 이전 은닉 벡터에 선형 변환을 적용하여 얻을 수 있음 그리고 이 과정은 시퀀스의 끝 (ex) 문장의 끝에 도달할 때까지 반복됨



RNN 학습 원리 : RNN은 시퀀스 데이터(순차 데이터)를 처리
 $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow \dots \rightarrow x_t$ 시간에 따라 들어오는 입력들을 하나씩 처리하고
→ hidden state를 다음으로 넘기며 → 마지막에 결과를 예측

순방향(Forward) : $x_1 \rightarrow h_1 \rightarrow y_1 / x_2 \rightarrow h_2 \rightarrow y_2 / x_3 \rightarrow h_3 \rightarrow y_3 / \dots / x_t \rightarrow h_t \rightarrow y_t$
입력 x_1 이 들어오면 hidden state h_1 생성 → h_1 은 다음 입력 x_2 와 합쳐져서 h_2 생성
이런 식으로 hidden state가 시간에 따라 이어지며 → 최종적으로 y_t 까지 결과가 나옴
이때, 매 시간마다 같은 가중치(W_{xh}, W_{hh}, W_{yh})가 사용(parameter sharing)

순방향(Forward)	역방향(Backward, BPTT)
시간 순서대로 입력 처리 ($x_1 \rightarrow x_t$)	시간 거꾸로 오차 전파 ($y_t \rightarrow y_1$)
hidden state 넘기기 ($h_{t-1} \rightarrow h_t$)	hidden state로 오차 전파 ($h_t \rightarrow h_{t-1}$)
모든 시점에 같은 가중치 W 사용	모든 시점의 가중치 사용 기록에 따라 gradient 계산
최종 결과 y_t 에서 Loss 계산	Loss를 통해 모든 가중치 업데이트

RNN 학습은 시간 축으로 펼쳐(Unroll)서 → 순방향은 시간 흐름대로 계산하고
→ 역방향은 시간을 거슬러 올라가며 오차를 전달하고 가중치를 업데이트하는 방식(BPTT)으로 이루어짐
→ 이 과정에서 시간이 길어질수록 gradient vanishing/exploding 문제가 발생하기 때문에
→ 이후 LSTM과 GRU가 필요하게 되었음

RNN 구조 유형

1. 일대다 (One-to-Many)

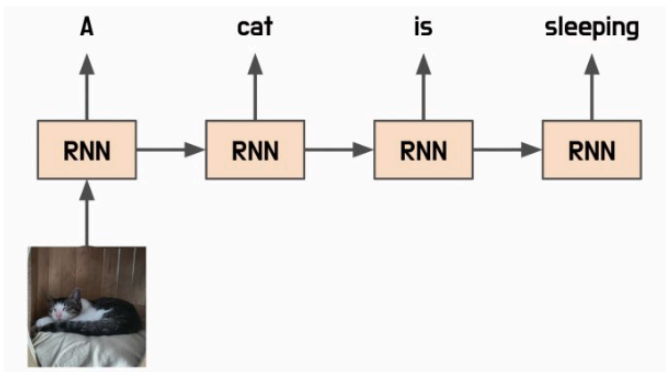


그림 6.13 순환 신경망의 일대다 구조

- 하나의 입력 → 여러 출력
- 예) 이미지 입력 → 문장(캡션) 생성

2. 다대일 (Many-to-One)

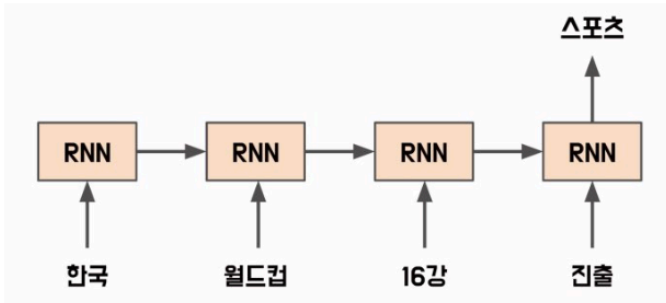


그림 6.14 순환 신경망의 다대일 구조

- 여러 입력 → 하나의 출력

- 예) 문장 입력 → 감정 분류 (긍정/부정)

3. 다대다 (Many-to-Many)

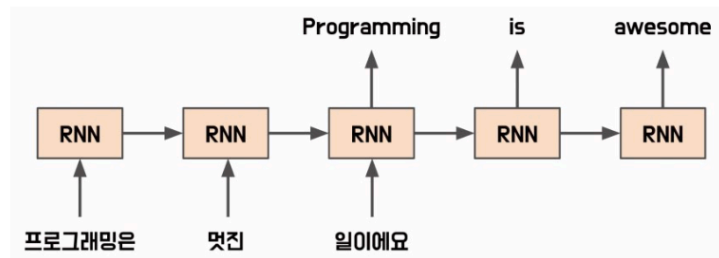


그림 6.15 순환 신경망의 다대다 구조

- 여러 입력 → 여러 출력
- 예) 문장 입력 → 번역된 문장 출력
- 입력/출력 길이가 다를 경우 **padding**, **Seq2Seq (Encoder-Decoder)** 구조 사용

양방향 RNN (Bidirectional RNN)

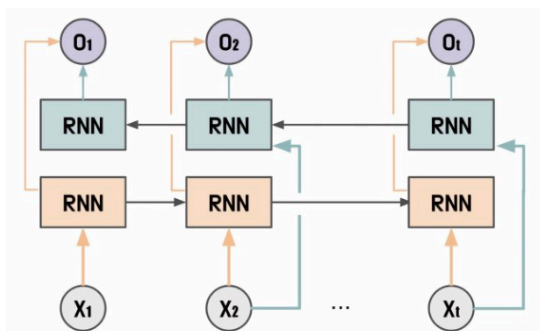


그림 6.16 양방향 순환 신경망 구조

- 각 시점의 은닉 상태를 계산할 때 과거(h_{t-1})뿐만 아니라 미래(h_{t+1})도 함께 고려
- 문장의 앞뒤 맥락 모두 활용 가능 → 예) "인생은 B와 C 사이의 D다" → D를 예측할 때 C 정보도 중요
- t시점 이후의 데이터도 t시점의 데이터를 예측하는 데 사용될 수 있음
 - 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로 거꾸로 읽어 들여 처리하는 방식으로 이루어짐
 - 대부분 연속형 데이터는 이전 시점 데이터뿐만 아니라 이후 시점의 데이터와 큰 상관관계 가짐
 - 그러므로 양방향 순환 신경망은 양방향적인 정보를 모두 고려하여 현재 시점의 출력값을 계산

다층 RNN (Stacked RNN)

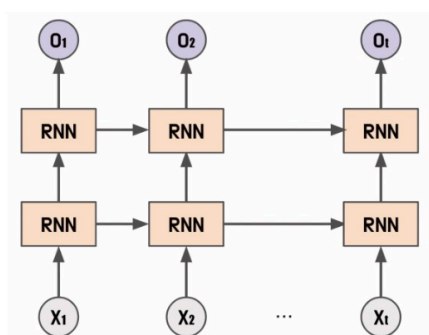


그림 6.17 다층 순환 신경망 구조

- 여러 개의 순환 신경망을 연결하여 구성한 모델 → 각 순환 신경망이 서로 다른 정보를 처리하도록 설계
- 장점: 더 복잡한 패턴 학습 가능
- 단점: 학습 시간 증가, 기울기 소실/폭주 문제

순환 신경망 클래스

```
nn.RNN(
    input_size,
    hidden_size,
    num_layers=1,
    nonlinearity="tanh",
    bias=True,
    batch_first=True,
    dropout=0,
    bidirectional=False
)
```

매개변수	설명
<code>input_size</code>	입력 벡터 크기
<code>hidden_size</code>	은닉 상태 크기
<code>num_layers</code>	층 수
<code>nonlinearity</code>	비선형 함수 (<code>tanh</code> , <code>relu</code>)
<code>bias</code>	편향 값 사용 유무
<code>batch_first</code>	입력 배치 크기를 첫 번째 차원으로 사용할 지 여부 True이면 입력 shape이 <code>[batch, seq, feature]</code>
<code>dropout</code>	과대 적합 방지
<code>bidirectional</code>	양방향 사용 여부

양방향 다층 신경망

```
import torch
import torch.nn as nn

input_size = 128
output_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size = input_size,
    hidden_size = output_size,
    num_layers = num_layers,
    nonlinearity = "tanh",
    batch_first = True,
    bidirectional = bidirectional
)

batch_size = 4
sequence_len = 6

inputs = torch.rand(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * int(bidirectional) + 1, batch_size, output_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape) # torch.Size([4, 6, 512])
print(hidden.shape) # torch.Size([6, 4, 256])
```

장단기 메모리 : LSTM (Long Short-Term Memory)

- RNN의 한 종류로 기존 RNN의 장기 의존성 문제와 기울기 소실 문제 해결
- 구조: Memory Cell + 3개의 Gate (Forget, Input, Output) → 정보의 흐름 제어

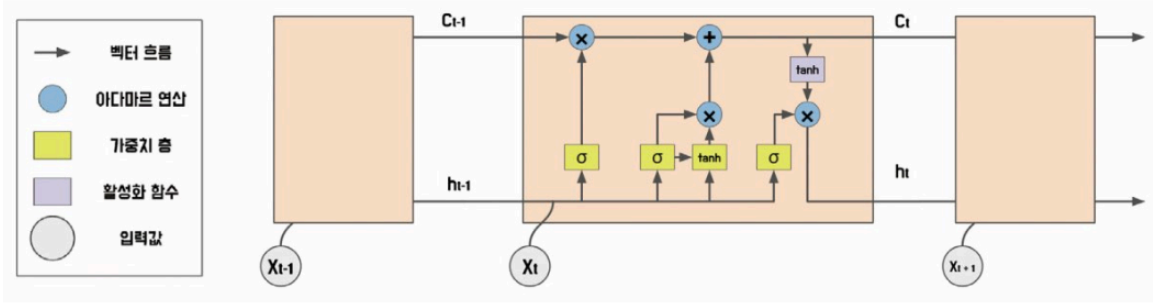


그림 6.18 장단기 메모리 구조

- 셀 상태는 중요한 정보를 장기적으로 저장하고, Gate들이 이 흐름을 조절함

LSTM 구조

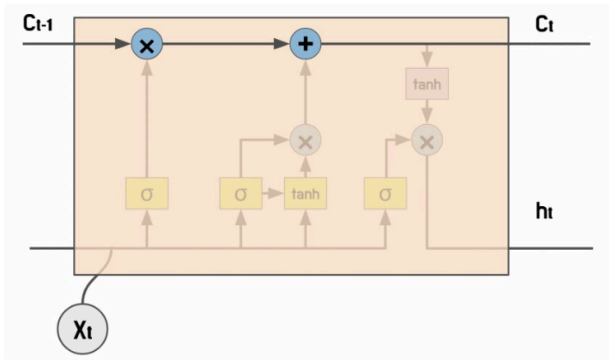


그림 6.19 장단기 메모리 셀 상태

- Memory Cell
 - 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보 계산 및 저장
 - 출력값 계산에 직접 사용되지 않음 (↔ RNN의 은닉 상태는 출력값 계산에 사용됨)

Gate 연산

Forget Gate : 셀이 이전 내부 상태에서 어떤 정보를 잊을지를 결정
 Input Gate : 어떤 새로운 정보를 내부 상태에 저장할지 알려줌
 Output Gate : 셀 상태에서부터 얼마나 많은 정보를 드러낼 것인지 결정

Long Short-Term Memory

Three gates exist to control information flow

이 LSTM의 핵심 아이디어는 정보를 제어하는 것
 -> 정보의 흐름을 제어하는 세 개의 게이트
 (ex) 방 안에 문이 있고 이 문을 열면 사람들이 들어올 수 있는 것처럼
 -> LSTM 셀 내부의 문을 열면 정보가 들어올 수 있음
 반대로 문을 닫으면 정보의 흐름이 차단됨
 -> LSTM 셀의 게이트를 blocked 하면 그 게이트를 통해서 정보가 흐르지 않음

- Key idea: use **gates** to control the flow of information
 - Forget gate** (f_t): tells the **cell** which information to forget from the internal cell state
 - Input gate** (i_t): tells the cell which new information to store in the internal cell state
 - Output gate** (o_t): what the cell outputs (a filtered version of the internal cell state)

- X_t : input
- h_t : output
- C_t : cell state
- $\hat{C}_t$: internal cell state

sigmoid 가중치 행렬 곱 이전 은닉상태, 현재 입력 벡터 concat 편향 더함
 0~1 사이의 값을 가짐

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_t] + b_i)$$

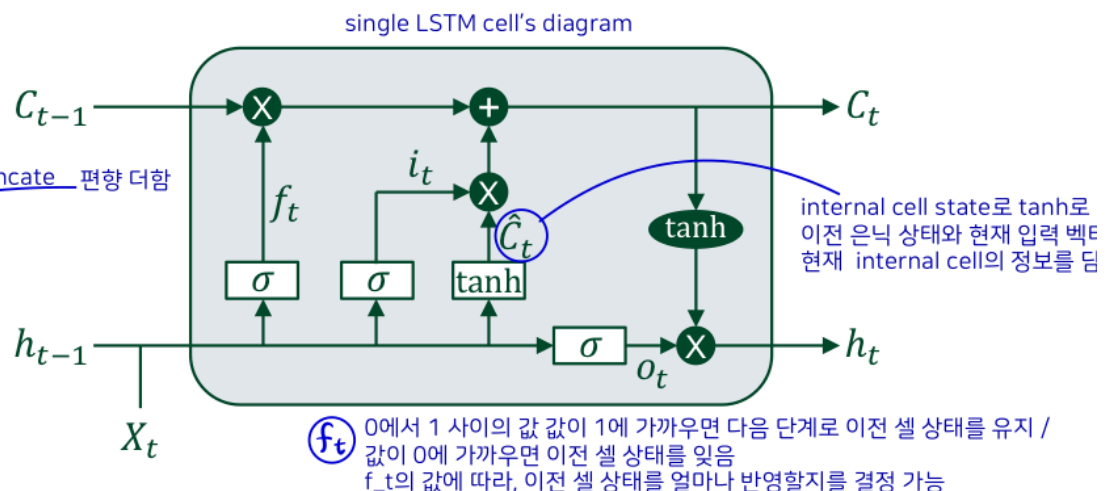
$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o)$$

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, X_t] + b_c)$$

게이트 역할을 하는 수식 2가지

$$C_t = i_t \cdot \hat{C}_t + f_t \cdot C_{t-1}$$

$$h_t = o_t \times \tanh(C_t)$$



C_t cell state, 일종의 장기기억(Long-Term Memory) 이 cell state 는 진짜 기억처럼 작동 (cf) 기존의 RNN에서는 hidden vector가 있었고 단기기억으로 사용되었음
 LSTM에서는 cell state(C_t)가 실제 기억
 해당 LSTM의 cell output -> 다음 LSTM으로 전달될 출력으로 셀 내부의 정보를 다음 단계로 전달

i_t 0에서 1 사이 값을 가지며, 1에 가까우면 $\hat{C}_t$ 더 많이 반영되고, 0에 가까우면 $\hat{C}_t$ 정보를 마스킹(무시)

h_t LSTM 셀의 최종 output 단기 기억에 해당 o_t 와 $\tanh(C_t)$ 를 곱한 값 -> 즉, 현재의 cell state는 o_t 가 1에 가까울 때 드러나게 되며, o_t 가 0에 가까울 때는 드러나지 않음 = output(hidden state)가 cell state에 대한 많은 정보를 담지 않게 됨

1. Forget Gate

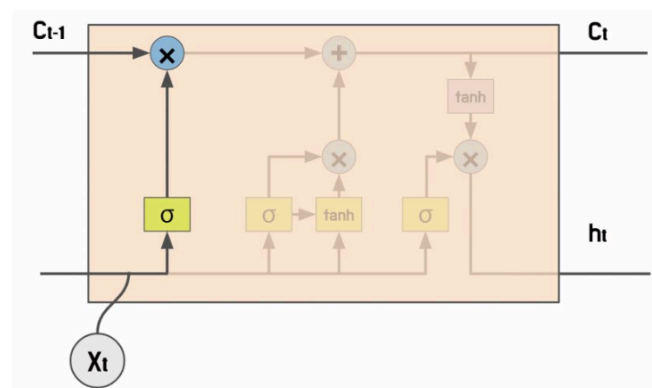


그림 6.20 장단기 메모리의 망각 게이트

$$f_t = \sigma(W_x^{(f)} x_t + W_h^{(f)} h_{t-1} + b^{(f)})$$

- 이전 셀 상태(C_{t-1})에서 무엇을 잊을지 결정

2. Input Gate + Cell Update

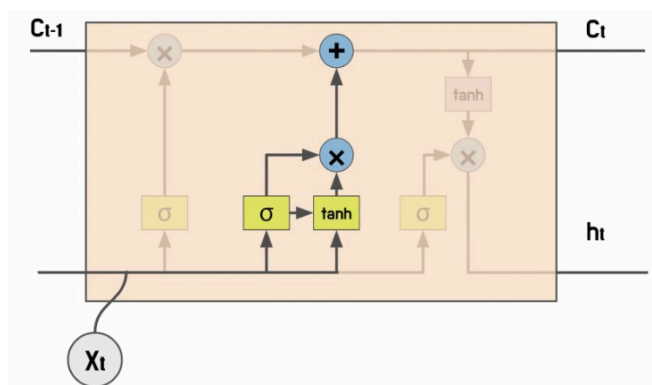


그림 6.21 장단기 메모리의 기억 게이트

$$g_i = \tanh(W_x^{(g)} x_t + W_h^{(g)} h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)} x_t + W_h^{(i)} h_{t-1} + b^{(i)})$$

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- 어떤 새로운 정보를 추가할지 결정

3. Output Gate

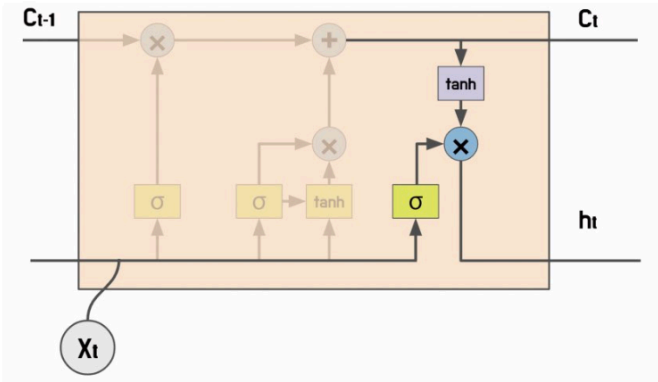


그림 6.22 장단기 메모리의 출력 게이트

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

- 어떤 정보를 은닉 상태로 출력할지 결정

LSTM 클래스

```
nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=True,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size=0
)
```

매개변수	설명
proj_size	은닉 상태를 투사하여 차원 축소 (0이면 비활성화)
hidden_size	셀 상태의 크기는 여전히 hidden_size 사용

양방향 다층 LSTM

```
import torch
import torch.nn as nn

input_size = 128
hidden_size = 256
proj_size = 64
num_layers = 3
bidirectional = True

model = nn.LSTM(
    input_size = input_size,
    hidden_size = hidden_size,
    num_layers = num_layers,
    batch_first = True,
```

```

    bidirectional = bidirectional,
    proj_size = proj_size
)

batch_size = 4
sequence_len = 6

inputs = torch.rand(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else output_size,
)
c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size)

outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

print(outputs.shape) # torch.Size([4, 6, 128]) = 2 directions x proj_size
print(hn.shape)      # torch.Size([6, 4, 64])
print(cn.shape)      # torch.Size([6, 4, 256])

```

- `outputs` shape은 `(batch, seq_len, num_directions x proj_size)`
- `h_n` 은 은닉 상태 (proj_size 기준), `c_n` 은 셀 상태 (hidden_size 기준)

합성곱 신경망(CNN)

- CNN은 이미지 인식 등 컴퓨터 비전 분야에서 주로 사용되는 인공지능망의 한 종류
- 입력 데이터의 **지역적인 특징(Local Feature)** 추출에 강하며, 합성곱 연산을 기반으로 함
- 입력 데이터의 각 지역 영역에 대해 연산 → **전역 특징(Global Feature)** 추출 가능
- 원래 컴퓨터 비전을 위해 개발되었지만, 이후 자연어 처리(NLP)에서도 효과적으로 사용됨

합성곱 계층

- 입력 데이터 + 필터(Filter)를 통해 합성곱 연산 수행 → 출력 데이터 생성
- 동일한 필터를 모든 위치에 적용하므로 매개변수 공유(Parameter Sharing) 가능 → 과적합 방지
- 필터 수가 많아질수록 다양한 특징 추출 가능 → 표현력 증가

필터(Filter)

- 일반적으로 3×3, 5×5 등의 작은 정방형 크기
- 필터의 가중치는 학습을 통해 갱신됨
- 필터를 입력 데이터에 슬라이딩하며 합성곱 연산 → 특징 맵(Feature Map) 생성

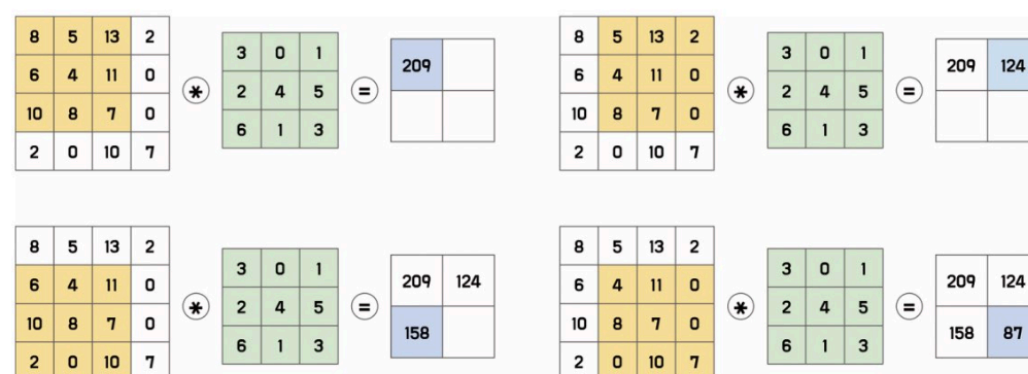


그림 6.24 특징 맵 추출 과정

- 예시) 특징 맵 생성

- 입력: 4×4 이미지
- 필터: 3×3
- Stride: 1 → 출력: 2×2 특징 맵
- 예시 수식: $(8 \times 3) + (5 \times 0) + (13 \times 1) + (6 \times 2) + \dots + (7 \times 3) = 209$

패딩(Padding)

- 합성곱 후 출력 크기가 줄어드는 문제를 해결하기 위해 가장자리에 0을 덧붙임 (Zero Padding)
- 입력과 출력 크기를 동일하게 유지 가능
- 가장자리 픽셀도 충분히 학습에 반영되도록 도움
- 제로 패딩

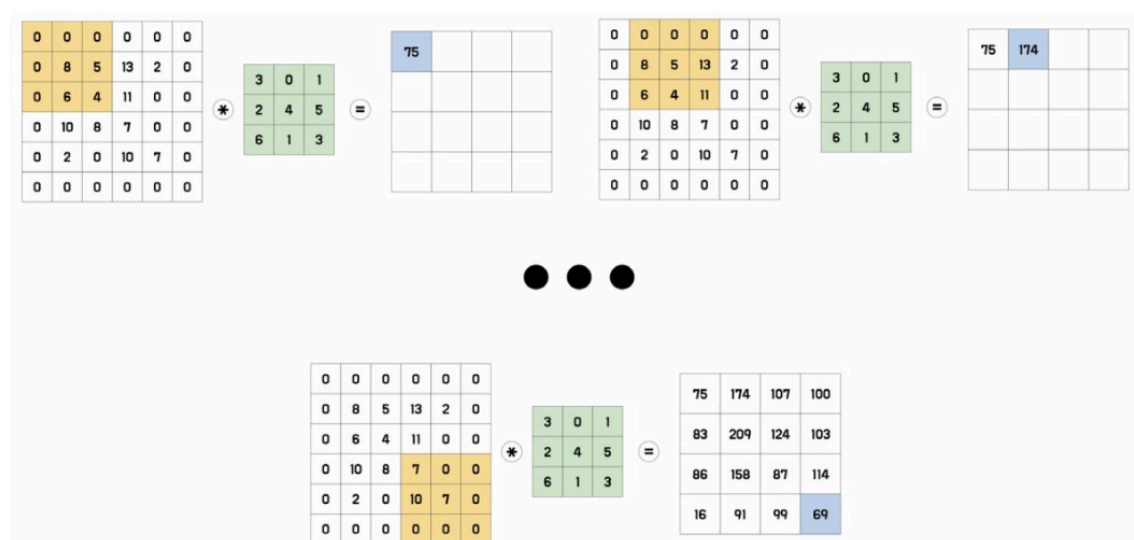


그림 6.25 제로 패딩

간격(Stride)

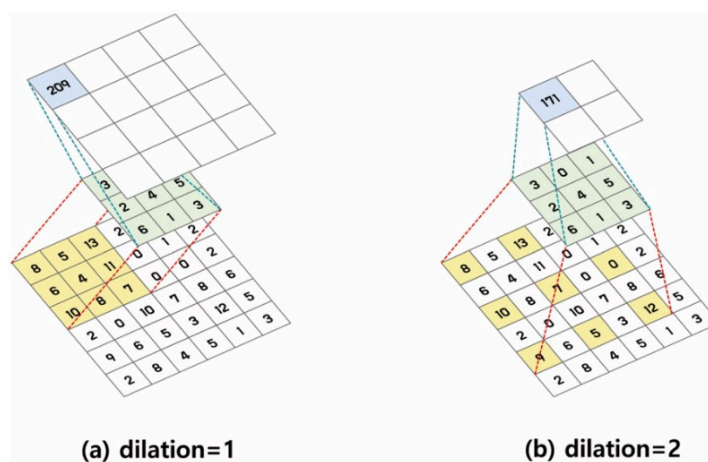
- 필터가 이동하는 간격
- Stride가 크면 출력 크기가 작아지고, 연산량 줄어들
- Stride가 작으면 공간 정보 유지, 크면 연산량 감소 & 과적합 방지 가능

채널(Channel)

- RGB 이미지처럼 3차원 입력에 대해 각 채널(R, G, B)에 대해 합성곱 수행
- 채널 수가 많아질수록 더 다양한 특징 추출 가능 → 표현력(Representational Power) 높아짐
- 그러나 매개변수 수가 증가해 학습 시간 증가 및 과적합 가능성 존재

팽창(Dilation)

- 필터 내부 요소 간에 공백을 뒀서 더 넓은 수용 영역 확보 → 필터와 입력 데이터 사이에 간격을 두는 방법
 - 일반적으로 팽창값은 1로 사용해 간격을 한 칸만 띄워 사용 → 이 값이 커질 수록 필터가 입력 데이터를 바라보는 범위가 넓어짐



- 필터 크기를 키우지 않고도 입력의 넓은 영역을 참조 가능 → 더 깊고 복잡한 모델 구성 가능
 - 팽창을 적절히 적용하면 모델의 매개변수 수를 줄일 수 있어 메모리 사용량 줄일 수 있음
 - 그러나 필터가 바라봐야 하는 입력 데이터의 범위가 커지므로 오히려 연산량이 늘어날 수 있음
 - 팽창 크기가 너무 크다면 인접한 픽셀값을 고려하지 않게 됨 → 공간적 정보가 보존되지 않아 특징 추출의 효과가 떨어질 수 있음

합성곱 계층 클래스

```
torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0,
    dilation=1,
    groups=1,
    bias=True,
    padding_mode="zeros"
)
```

- `in_channels`, `out_channels`: 채널 수
- `kernel_size`: 필터 크기
- `groups`: 채널을 여러 그룹으로 나눠 합성곱 수행 (파라미터 수 절감 가능)
- 출력 크기 계산 공식

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

활성화 맵(Activation Map)

- 합성곱 계층의 특징 맵에 활성화 함수(ReLU 등 비선형 함수)를 적용해 얻어진 출력값
 - ReLU: 0보다 크면 그대로, 아니면 0으로
- 이렇게 얻은 활성화 맵은 다음 계층의 입력으로 사용
 - 다음 계층이 합성곱 계층이라면 동일한 방식의 연산이 수행돼 합성곱 연산과 활성화 함수를 거치게 됨
 - 합성곱 연산과 활성화 함수를 여러 번 반복하여 신경망 구성 시 → 입력 이미지에서 추출된 추상적인 특징을 학습할 수 있음
- 비선형성 도입 → 복잡한 패턴 학습 가능
 - 합성곱 계층의 출력값에 활성화 함수를 적용하지 않으면 합성곱 연산 결과값이 그대로 다음 계층으로 전달됨 → 선형적 결합만 수행 → 복잡 패턴 추상 특징 학습 불가
 - 따라서, 활성화 함수를 적용해 모델이 비선형성을 가지게 되어 입력 데이터에서 다양한 추상적인 특징을 학습할 수 있게 됨

풀링(Pooling)

- 특징 맵 크기를 줄이는 작업 → 합성곱 계층 다음에 적용됨
- 연산량 감소, 입력 데이터 압축
- 대표 방식:
 - Max Pooling: 가장 큰 값 선택

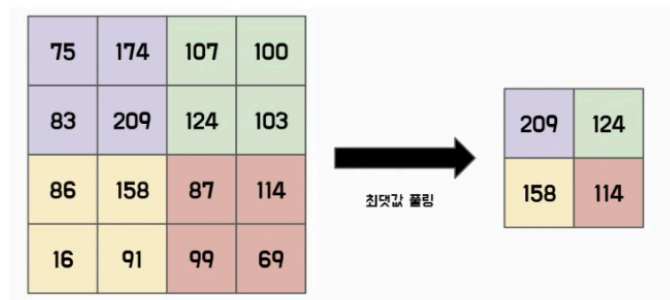


그림 6.27 최대값 풀링

- Avg Pooling: 평균값 선택

풀링(Pooling) 클래스

- 2차원 최대값 풀링 클래스

```
pool = torch.nn.MaxPool2d(
    kernel_size,
    stride=None,
    padding=0,
    dilation=1
)
```

- 2차원 평균값 풀링 클래스

```
pool = torch.nn.AvgPool2d(
    kernel_size,
    stride=None,
    padding=0,
    count_include_pad=True
)
```

- `count_include_pad` : 패딩 영역의 값을 평균 계산에 포함할지 여부 설정
 - True 설정 시 : 제로 패딩의 값이 평균값 풀링 연산에 포함됨

완전 연결 계층(Fully Connected Layer, FC)

- 각 입력 노드가 모든 출력 노드와 연결된 상태 → FC layer는 입력과 출력 간의 모든 가능한 관계 학습 가능
- 특징 맵 → 1D 벡터로 펼친 뒤 선형 변환 수행
- 마지막 분류기 역할 수행 (예: `nn.Linear(32*32*32, 10)`)
- 모든 노드가 연결 → 추상적인 정보 조합 가능
- 소프트맥스나 시그모이드와 같은 출력 활성화 함수와 결합해 분류 작업 수행

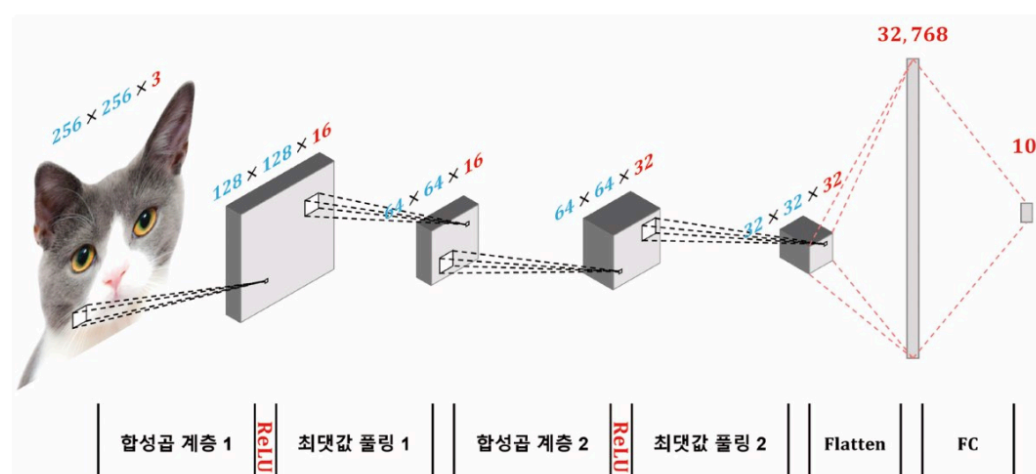


그림 6.28 합성곱 모델 구조 시각화

합성곱 모델

```

import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.fc = nn.Linear(32 * 32 * 32, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = torch.flatten(x)
        x = self.fc(x)
        return x

```